الرسم بالحلقات: for المتداخلة

الدرس 13 — 1IOT، SST-ICT-01 — تمرين: النجوم، المستطيل الممتلئ والفارغ، المثلث

20/10/2026

1 ماذا فعلنا اليوم

استخدمنا اليوم بنى الدرس 12 (**for** و**while** و**do** **while**) في **سلسلة** **من** **التمارين**، كل منها أصعب قليلًا من سابقه: لا يوجد أبدًا تمرين **أسهل** مما قبله. نبدأ بسطر من النجوم، ونصل إلى **مستطيل**، ثم مستطيل **فارغ** وأخيرًا **مثلث**. وفي كل التمارين توجد أيضًا مشكلة **التحقق** **من** **المعلومة** التي يدخلها المستخدم.

- يُكتب كل تمرين في **مشروع** **جديد** على onlinegdb: أما ما أُنجز **فلا** **يُعدَّل**، بل يُحتفظ به.
- قبل المتابعة **جرّب** ما كتبته، حتى لو لم ينته البرنامج: هكذا تجد الأخطاء فورًا، والكود لا يزال قصيرًا.
- يجب أن يكون الكود دائمًا **بإزاحة** (القسم 4.4).

ملاحظة

الحرف المطبوع لا يهم: تُستخدم **النجمة** (**asterisco**) بحكم العادة، لكن x أو @ أو أي حرف آخر يصلح تمامًا.

2 التمرين 1: سطر من النجوم

مثال

النص. اكتب برنامجًا يطلب من المستخدم إدخال عدد **أكبر** **من** **0**. ثم يطبع البرنامج **عدد** **نجوم** يساوي ذلك العدد.

2.1 يجب التحقق من المعلومة

لإدخال معلومة إلى الآلة **أثناء** **تنفيذ** البرنامج (يقال **a runtime**) نحتاج، كالعادة، إلى `printf` يقول للمستخدم ماذا نريد و`scanf` **يحصل** على المعلومة.

الجديد أن العدد يجب أن يكون **أكبر** **من** **0**: 1، 2، 3، 4... مقبولة، و0 و-1 و-5 **غير** مقبولة. إذا كتب المستخدم معلومة لا تحترم المطلوب، **لا** **تقبلها** الآلة و**تعيد** **السؤال**.

ما البنية الأنسب؟ **do** **while**، لأن السؤال يجب أن يُطرح **مرة** **واحدة** **على** **الأقل**. يقول شرط `while` **متى** **نكرر**، أي متى تكون المعلومة **خاطئة**: نكرر ما دام `N < 1`.

تنبيه

لماذا `N < 1` وليس `N < 0`؟ مع `N < 0` يُقبل **الصفر** (0 ليس أصغر من 0)، ونحن **لا** **نريد** الصفر. الكتابتان الصحيحتان **متكافئتان**: `N < 1` أو `N <= 0`. نختار إحداهما.

يمكن أن تكون while الأخيرة على سطر القوس المغلق نفسه أو على سطر جديد: لا يهم لغة C كيف تنتقل إلى سطر جديد أو كيف تزيح، فالبرنامج يعمل على أي حال. يمكنك كتابة كل شيء على سطر واحد، لكنه عندئذ لا يُقرأ. وداخل قوسي do تذكر دائمًا فراغ الإزاحة.

2.2 نجرب الجزء الأول فورًا

قبل كتابة حلقة النجوم يُستحسن تشغيل (Run) ما هو موجود، لنرى هل توجد أخطاء. يطلب البرنامج العدد؛ وبالطبع لا يطبع النجوم بعد، لكن يمكننا تجربة السؤال (**Inserisci il numero di asterischi da stampare** = أدخل عدد النجوم المراد طباعتها):

```
Inserisci il numero di asterischi da stampare: 0
Inserisci il numero di asterischi da stampare: -5
Inserisci il numero di asterischi da stampare: 5
```

مع 0 و5− يتكرر السؤال: المعلومة غير مقبولة. ومع 5 يخرج البرنامج من الحلقة ويصل إلى النهاية. إلى هنا كل شيء صحيح.

2.3 النجوم: for

الآن يجب طباعة N من النجوم. عدد التكرارات نعرفه مسبقًا (هو N)، فنستخدم for. نحتاج إلى متغير جديد للعد: لا يمكن أن يكون N، لأننا نستخدم N حدًا (إلى أين نصل). نسميه x: يبدأ من 0، ويتكرر ما دام x < N، وفي كل دورة x++.

يمكن تعريف متغير الحلقة مباشرة داخل for، كما نفعل في القسم: for (int x = 0; x < N; x++). وهكذا يوجد x داخل الحلقة فقط، ولا حاجة إلى تعريفه في أعلى البرنامج.

لكن for المكتوبة هكذا لا تفعل شيئًا بعد: قلنا لها كم مرة تكرر، لكن ليس ماذا تكرر. في الداخل توضع طباعة نجمة، printf("*");.

```c
#include <stdio.h>

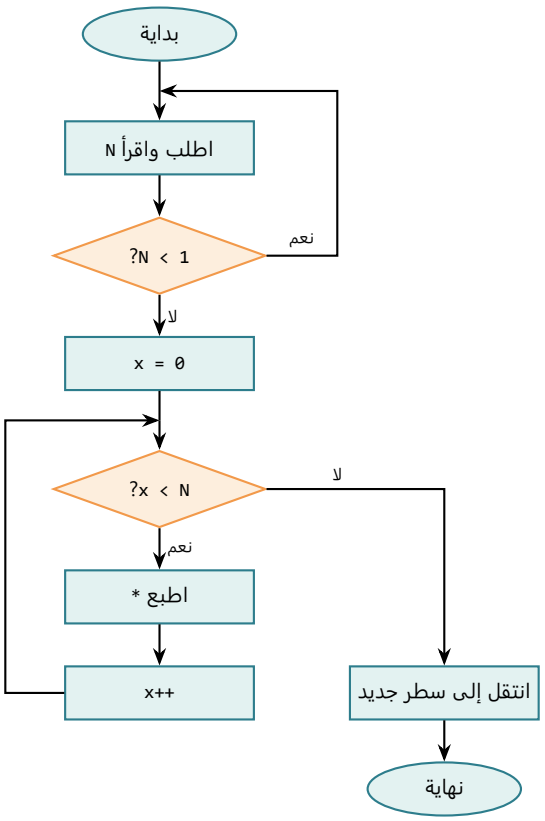
int main()
{
    int N;

    // أطلب العدد حتى يصبح أكبر من 0
    do
    {
        printf("Inserisci il numero di asterischi da stampare: ");
        scanf("%d", &N);
    } while (N < 1);

    // أطبع N من النجوم على السطر نفسه
    for (int x = 0; x < N; x++)
    {
        printf("*");
    }
    // سطر جديد أخير (أضيف في الملزمة: لا يبقى المؤشر على السطر)
    printf("\n");

    return 0;
}
```

```
Inserisci il numero di asterischi da stampare: 5
*****
```



شكل 1: التمرين 1: `do while` للتحقق من المعلومة، ثم `for` للنجوم.

2.4 مراجعة: do while و while

do while شكل من `while` يضمن تنفيذ الكود بين القوسين **مرة واحدة على الأقل**. أما `while` فتفحص الشرط **أولًا**: إذا كان خاطئًا من أول مرة، **لا يُنفذ** الكود **ولا مرة**، فلا يُطرح السؤال على المستخدم أبدًا. لهذا نستخدم `do while` لطلب معلومة.

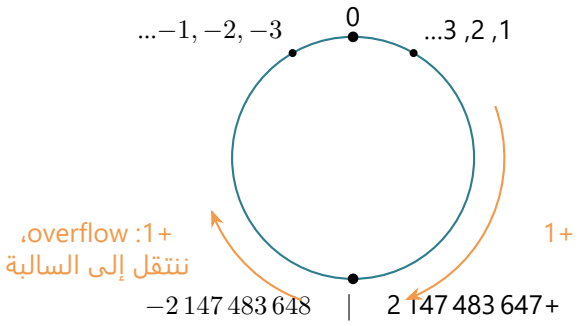
2.5 سؤال من القسم: وإذا كان العدد كبيرًا جدًا؟

عند تجربة البرنامج بأعداد **كبيرة جدًا** تحدث أشياء غريبة: أحيانًا يعيد البرنامج السؤال كأن العدد سالب، وأحيانًا يبدأ طباعة النجوم **دون توقف**. السبب هو **نوع البيانات**.

لـ`int` **حجم بالبتات**: تضمن C **16 بت على الأقل**، لكن الحجم الحقيقي يعتمد على الآلة. في الحواسيب الحالية (وعلى onlinegdb) يكون `int` **32 بت**. و32 بت تكون التركيبات 2^{32}، أي نحو **4 مليارات** (4 294 967 296). وبما أن `int` يحتوي أعدادًا **موجبة وسالبة**، ينقسم هذا المجال نصفين: من نحو -2 مليار إلى نحو $+2$ مليار (بالضبط من $-2\,147\,483\,648$ إلى $+2\,147\,483\,647$).

تعريف

الأعداد الصحيحة في الحاسوب مرتبة كما على **دائرة**: نبدأ من 0، ونصعد 1، 2، 3... وفي منتصف الدائرة نصل إلى الحد الأقصى، نحو **+2 مليار**. وعلى الجهة الأخرى السالبة، -1، -2، -3... حتى نحو -2 **مليار**. إذا أضفت 1 إلى أكبر عدد **لا** أحصل على عدد أكبر: بل أنتهي عند -2 مليار. وهذا خطأ رياضيًا: إنه **overflow** (الفيضان).



كان overflow قديمًا مشكلة **نصطدم بها باستمرار**؛ ومع آلات اليوم يصعب الوصول إليه، لكنه **ليس مستحيلًا**. في برنامجنا:

- مع **3 مليارات** يحتوي `int` عددًا **سالبًا** ($-1\,294\,967\,296$): يعتبره `do while` خاطئًا و**يعيد السؤال**؛
- مع **5 مليارات** يحتوي `int` عددًا **موجبًا** ($705\,032\,704$): يقبله البرنامج ويطبع **أكثر من 700 مليون** نجمة، ويبدو أنه لا يتوقف أبدًا.

ملاحظة

للتعمق. رأينا في القسم أنه مع عدد يتجاوز 2 مليار يعيد البرنامج السؤال، كأنه سالب. وقد جُربت الحالتان أعلاه (3 و5 مليارات) على الحاسوب بعد الدرس، لشرح حالة البرنامج الذي "لا يتوقف" أيضًا. في معيار C، تجاوز حد `int` ذي الإشارة **سلوك غير محدد**: أما "الدوران على الدائرة" فهو ما تفعله حواسيبنا عمليًا. لا حاجة لتذكر القيمة الدقيقة: المهم أن نعرف أن لكل **نوع بيانات حدًا**.

2.6 وإذا كتبت حرفًا؟

في `scanf` كتبنا `%d`: قلنا له أن يحصل على **عدد صحيح**. إذا كتب المستخدم **حرفًا** (أو اسمه)، **لا يستطيع** `scanf` قراءته، فلا يتلقى المتغير قيمة صالحة ويبقى الحرف هناك غير مقروء: قد يعيد البرنامج السؤال **إلى ما لا نهاية** أو يتصرف بغرابة. حاليًا نعرف الحصول على **الأعداد فقط**؛ أما **الكلمات** فتحتاج إلى أدوات سنراها لاحقًا، تقريبًا في **نوفمبر**.

3 التمرين 2: مستطيل النجوم

مثال

النص. اكتب برنامجًا يطلب من المستخدم إدخال **عددين** **أكبر** **من** **0**، `C` و`R`. يطبع البرنامج **مستطيلًا** من النجوم، **عرضه** `C` و**ارتفاعه** `R`.
مثال: إذا كان `C` يساوي 5 و`R` يساوي 3، يطبع البرنامج

```
*****
*****
*****
```

تساعد الأسماء على التذكر: `C` تعني **colonne** (الأعمدة، العرض)، و`R` تعني **righe** (الأسطر، الارتفاع).

3.1 السؤالان

أولًا نعرّف المتغيرين، `int C` و`int R`. ثم السؤالين، لكل منهما **while** **do** خاصة (نكرر ما دام العدد أصغر من 1)، تمامًا كما في التمرين 1.

تنبيه

ضع الطلب **في** **سياقه**. يمثل العدد الأول **عرض** (**larghezza**) المستطيل: إذن لا يقول `printf` "أدخل العدد الأول"، بل `"Inserisci la larghezza del rettangolo"`. والثاني: `"Inserisci l'altezza del rettangolo"`. يجب أن يفهم المستخدم **ماذا** تطلب منه.

لا توضع `for` أثناء طلب البيانات: ففي تلك اللحظة لا نعرف **كم** **مرة** نكرر. تُكتب **بعد** الحصول على المعلومتين.

3.2 تقسيم المسألة إلى قسمين

طباعة مستطيل أكثر تعقيدًا قليلًا من التمرين 1. يُستحسن **تقسيم** **المسألة** إلى جزأين.

1. **الجزء الأول:** طباعة **سطر** من `C` نجمة. هذا نعرفه: إنه التمرين 1، `for` فيها `x` من 0 ما دام `x < C`، وفي داخلها `printf("*");`.
2. **الجزء الثاني:** يجب طباعة هذا السطر `R` **مرة** (في المثال 3). يمكنني كتابة `for` نفسها ثلاث مرات بالنسخ واللصق، لكن ماذا لو كان `R` يساوي 50؟ **لا** **نفعل** **هكذا**: توضع كتلة `for` كلها **داخل** `for` **أخرى**، تكررها `R` مرة. نحتاج إلى **متغير** **مختلف**: `y`، من 0 ما دام `y < R`.

هكذا يكرر البرنامج **3** **مرات** شيئًا يكرر **5** **مرات**. هل تتذكر ما اسم `if` داخل `if` أخرى؟ **متداخلة** (**annidato**). وبالمثل تسمى `for` داخل `for` **متداخلة** (أو حلقة متداخلة). `for` التي فيها `y` هي الحلقة **الخارجية** (الأسطر)، والتي فيها `x` هي الحلقة **الداخلية** (الأعمدة).

3.3 المحاولة الأولى: شيء ناقص

عند كتابة `for` مرتين فقط، إحداهما داخل الأخرى، مع `printf("*");` في الداخلية، يكون الكود **مكتوبًا** **بشكل** **صحيح** (لا أخطاء)، لكنه **لا** **يفعل** **بعد** ما نريد: مع 5 و3 لا يخرج مستطيل، بل **سطر** **واحد** من 15 نجمة.

```
***************
```

ماذا ينقص؟ في كل مرة ننتهي من كتابة **سطر** يجب أن **ننتقل** **إلى** **سطر** **جديد**. ليس عند كل نجمة، بل **كل** **5** **نجوم**، أي عندما تنتهي الحلقة الداخلية. إذن يوضع `printf("\n");` **بعد** **for** **الداخلية**، لكن **داخل** **for** **الخارجية**.

```c
#include <stdio.h>

int main()
{
    int C;
    int R;

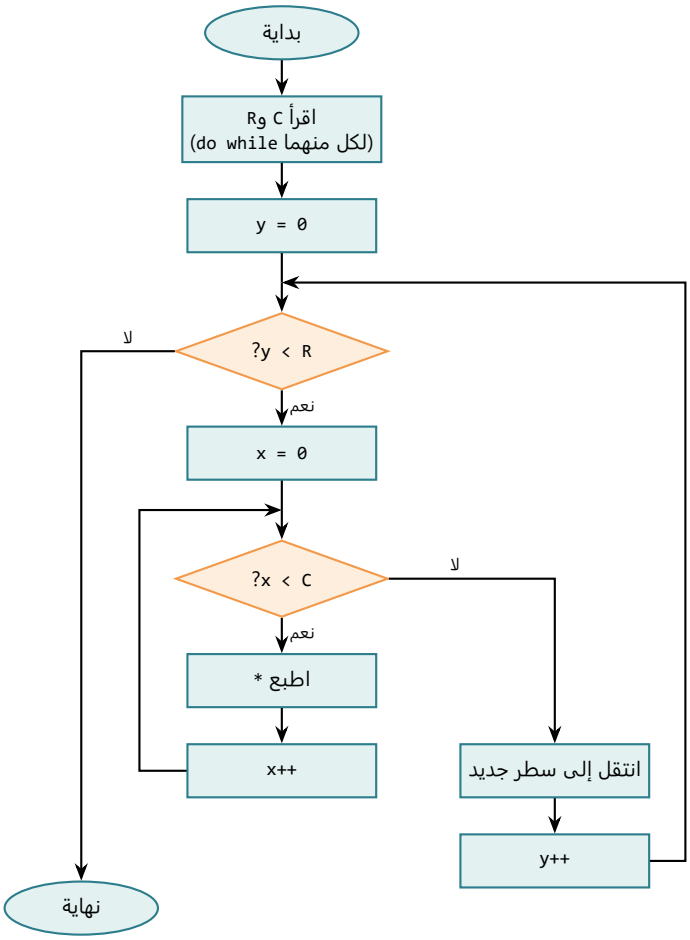
    // عرض المستطيل (الأعمدة)
    do
    {
        printf("Inserisci la larghezza del rettangolo: ");
        scanf("%d", &C);
    } while (C < 1);

    // ارتفاع المستطيل (الأسطر)
    do
    {
        printf("Inserisci l'altezza del rettangolo: ");
        scanf("%d", &R);
    } while (R < 1);

    // الحلقة الخارجية: مرة لكل سطر
    for (int y = 0; y < R; y++)
    {
        // الحلقة الداخلية: سطر من C نجمة
        for (int x = 0; x < C; x++)
        {
            printf("*");
        }
        // انتهى السطر، سطر جديد
        printf("\n");
    }

    return 0;
}
```

```
Inserisci la larghezza del rettangolo: 5
Inserisci l'altezza del rettangolo: 3
*****
*****
*****
```



شكل 2: التمرين 2: حلقتان متداخلتان. تطبع الحلقة الداخلية (x) سطرًا؛ وتكرره الخارجية (y) R مرة، مع الانتقال إلى سطر جديد بعد كل سطر.

4 التمرين 3: المستطيل الفارغ

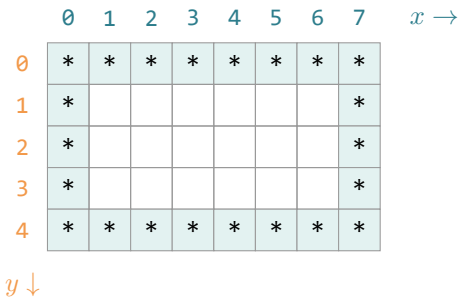
بعد المستطيل الممتلئ جاء طلب من القسم: **وماذا لو أردته فارغًا**، أي بنجوم **على الحافة فقط**؟ هذا هو الطريق الصحيح: كل يوم سنكتب أكوادًا **أكثر تعقيدًا**. لنجرب.

مثال

النص. عدّل برنامج التمرين 2 (عُمل في القسم على البرنامج نفسه) ليكون المستطيل **فارغًا**: تُطبع النجوم **على الحافة فقط**، وفي الداخل **فراغات**.

4.1 الإحداثيات، كما في لعبة battaglia navale

لنرسم مستطيلًا أكبر، **8** **في** **5** (`C` = 8، `R` = 5)، ونعطي رقمًا لكل عمود وكل سطر، كما في لعبة **battaglia navale** (معركة السفن). الأعمدة هي x، من 0 إلى 7 نحو اليمين؛ والأسطر هي y، من 0 إلى 4 نحو الأسفل. نبدأ من **0** لأن `for` عندنا تبدأ من 0.



متى تُطبع النجمة؟

- عندما تكون y مساوية **0**: يُطبع **السطر الأول** كله؛
- عندما تكون x مساوية **0**: يُطبع **العمود الأول** كله؛
- في **السطر الأخير**. كم تساوي y في السطر الأخير؟ في الرسم 4، لكن **ليس دائمًا** 4: إذا كان الارتفاع 5 فالسطر الأخير 4، وإذا كان 10 فهو 9، وإذا كان 18 فهو 17. وبشكل عام السطر الأخير هو **`R - 1`**، لأن العد يبدأ من 0؛
- في **العمود الأخير**، للسبب نفسه: **`x == C - 1`**.

إذن يوضع `printf` النجمة، في الحلقة الداخلية، تحت `if`: يطبع إذا كان `x == 0` **أو** `y == 0` **أو** `y == R - 1` **أو** `x == C - 1`. و"أو" هي العامل **OR**، `||`.

4.2 شرط في كل مرة

مع الشرطين الأولين (`x == 0 || y == 0`) يطبع البرنامج فعلًا السطر الأول والعمود الأول. لنضف السطر الأخير، `y == (R - 1)`، ونجرب بـ8 و5:

```
********
*
```

```
*
*
********
```

جاء السطر السفلي أيضًا. ماذا ينقص؟ **العمود الأخير**: `x == (C - 1)`.

4.3 المحاولة بلا else: تتكدس النجوم

عند إضافة العمود الأخير أيضًا تكون النتيجة **غير ما نريد**: المنطق سليم، والآلة فعلت **بالضبط ما كتبناه**، لكن العمود الأخير **ليس** في مكانه: تنتهي النجوم **ملتصقة** بنجوم العمود الأول.

```
********
**
**
**
********
```

المشكلة أننا نسينا أن نقول ماذا نفعل عندما تكون **كل الشروط خاطئة**. إذا لم نقل شيئًا **لا يطبع** البرنامج **شيئًا**: يطبع نجمة العمود الأول، وعندما تصل x إلى 7 يطبع الثانية **بجانبها مباشرة**. بينما يجب، في كل مرة لا تتحقق فيها الشروط، أن يطبع **فراغًا**: الفراغ لا يُرى، لكنه **يشغل المكان** ويسمح لنجمة العمود الأخير بالوصول إلى حيث نريد. تكفي إضافة **else** تطبع فراغًا.

```
    for (int y = 0; y < R; y++)
    {
        for (int x = 0; x < C; x++)
        {
            // نجمة على الحواف فقط، وإلا فراغ
            if (x == 0 || y == 0 || y == (R - 1) || x == (C - 1))
            {
                printf("*");
            }
            else
            {
                printf(" ");
            }
        }
        printf("\n");
    }
```

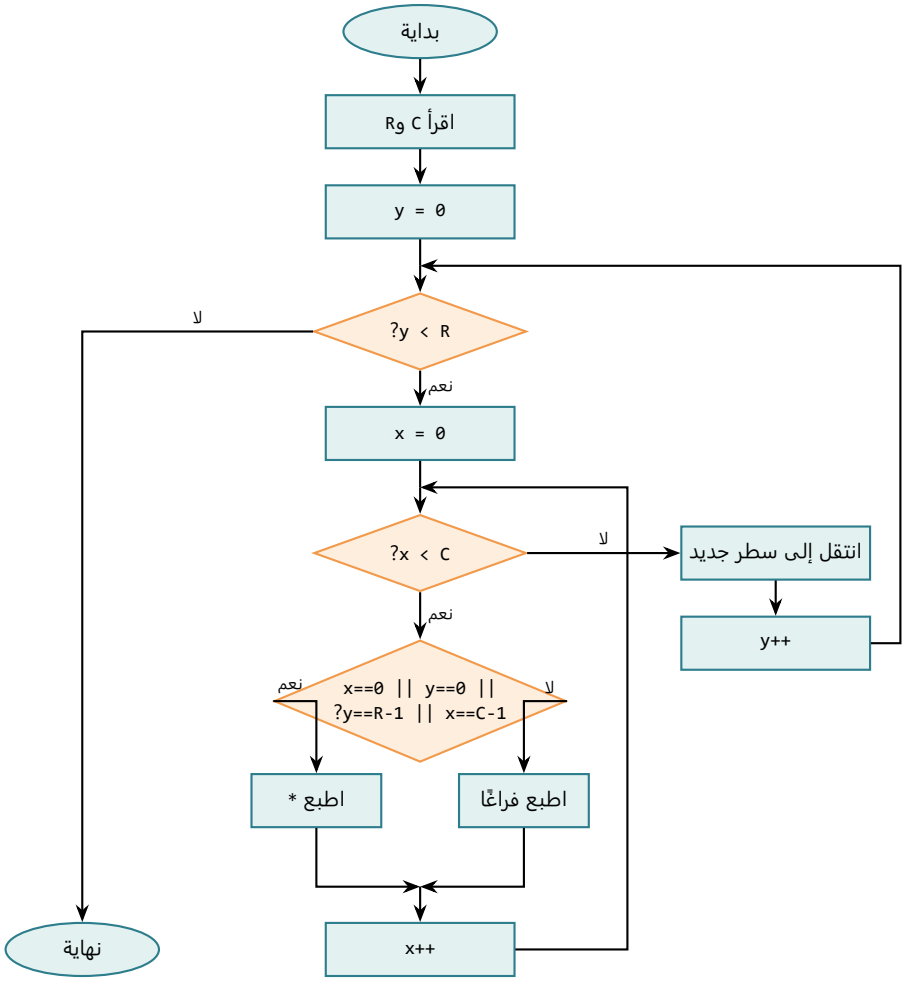
الأقواس حول `R - 1` و`C - 1` ليست إلزامية (المقارنة == تتم بعد الطرح)، لكنها تجعل الشرط **أسهل قراءة**: وقد وضعناها في القسم.

```
Inserisci la larghezza del rettangolo: 8
Inserisci l'altezza del rettangolo: 5
********
*      *
*      *
*      *
********
```

ملاحظة

لرؤية الفراغات. جرب طباعة **نقطة** بدلًا من الفراغ، `printf(".");`: يُطبع المستطيل والنقاط في وسطه. تلك النقاط هي **الأماكن الفارغة** التي لا نريد ملأها: لهذا يوجد في مكانها فراغ في النسخة النهائية.

```
********
*......*
*......*
*......*
********
```



شكل 3: التمرين 3: داخل الحلقة الداخلية تختار `if ... else` بين النجمة (الحافة) والفراغ (الداخل).

4.4 الإزاحة تفيد في إيجاد الأخطاء

في هذا التمرين الأقواس كثيرة: `for` مرتين، و`if`، و`else`. إذا كُتب الكود **كله** **مستقيمًا**، بلا إزاحة، لا يُفهم شيء، ويصبح إيجاد خطأ (قوس ناقص، فاصلة منقوطة زائدة) صعبًا جدًا، **حتى** **على** **الأستاذ**. القواعد:

- كلما **فُتح** **قوس** ينتقل الكود الداخلي إلى اليمين **جدولة** **واحدة**؛
- يوضع **القوس** **تحت** الكلمة التي تفتحه: تحت `f` من `for`، وتحت `i` من `if`؛
- `printf` داخل `if` داخل `for` مرتين يُزاح **ثلاث** **جدولات** (إضافة إلى جدولة `main`).

يرسم محرر onlinegdb **خطوطًا** **عمودية** تبين أين **تبدأ** **وتنتهي** الكتلة: ومع الكود المزاح يُرى فورًا هل يوجد قوس ناقص. أخطاء معتادة وُجدت بهذه الطريقة: قوس فتح `for` ناقص؛ `printf("\n");` **في** **غير** **مكانه** (يوضع بعد الحلقة الداخلية، لا داخلها)؛ `return 0;` **بعد** القوس الأخير بدلًا من قبله؛ فاصلة منقوطة بعد قوس `for` أو `if`، تفرغ الحلقة؛ `x` مكتوبة بدلًا من `y`.

ملاحظة

هذه الرسوم بالحروف هي أساس **الرسومات**: في السنة القادمة، بدلًا من النجوم، سنضيء **بكسلات**، وبتحريكها يمكن صنع رسوم متحركة وألعاب فيديو بسيطة مثل **Pong** (المضربان والكرة). أما لعبة مثل Pac-Man فأكثر تعقيدًا، خصوصًا في الرسومات.

5 التمرين 4: المثلث القائم

مثال

النص. اكتب برنامجًا يطلب من المستخدم إدخال عدد `N` **أكبر** **من** **0** و**أصغر** **من** **10**. يطبع البرنامج **مثلثًا** **قائمًا** من النجوم ضلعه `N`.
مثال: إذا كان `N` يساوي 5، يطبع البرنامج

```
*
**
***
****
*****
```

المثلث **القائم** هو مثلث فيه **زاوية** **90** **درجة**. وهذا أيضًا **متساوي** **الساقين**: له ضلعان متساويان، طولهما `N`.

تنبيه

احترم المطلوب. يجب أن يفعل البرنامج **ما** **يطلبه** **النص**: أن يطلب من المستخدم عددًا ويطبع المثلث بذلك الضلع. البرنامج الذي يطبع مثلثًا ثابتًا بكثير من `printf`، وربما منسوخًا من مثال على الإنترنت، **ليس** **طريقة** **أخرى** للحل: بل هو **شيء** **آخر**. إذا طلبت في مطعم البيتزا margherita وأحضروا لك قطعة سلمون، وجعلوك تدفع الثمن نفسه، هل يرضيك ذلك؟ الاستلهام من مثال لا بأس به، لكن يجب **تكييفه** مع المطلوب.

5.1 المشكلة الأولى: العدد بين 1 و9

"أكبر من 0 وأصغر من 10" يعني أن الأعداد المقبولة **من** **1** **إلى** **9**: 0 غير مسموح، وكذلك 10.

يقول شرط `do while` **متى** **نكرر**، لا متى تكون المعلومة صحيحة. يتم التفكير في خطوتين:

1. نكتب متى يكون العدد **صحيحًا**: `N > 0 && N < 10`. أي "أكبر من 0 **و**أصغر من 10": و"و" هي العامل **AND**، &&؛
2. لكن يجب أن نكرر عندما يكون العدد **غير** صحيح: نضع الشرط كله بين قوسين وأمامه **علامة** **التعجب**، **NOT**، التي **تنفي**: تجعل الصحيح خطأ والخطأ صحيحًا.

```
    do
    {
        printf("Inserisci il numero: ");
        scanf("%d", &N);
    } while ( !((N > 0) && (N < 10)) );
    // نكرر إذا لم يكن العدد أكبر من 0 وأصغر من 10
```

تنبيه

خطأ معتاد: OR بدلًا من AND. كتابة `N > 0 || N < 10` ("أكبر من 0 **أو** أصغر من 10") خطأ: **كل** عدد أكبر من 0 أو أصغر من 10 (حتى -5، وحتى 42)، إذن هذا الشرط **صحيح** **دائمًا**. مع OR **يستمر** البرنامج **في** **السؤال** أو يقبل كل شيء: ولا يفعل ما يطلبه النص. تذكر: "أو" هي **OR**، و"و" هي **AND**. وما اسم العامل الذي يفعل **العكس**، فيجعل الصحيح خطأ والخطأ صحيحًا؟ إنه NOT. علامة التعجب **ليست** "NOR": إنها **NOT**.

لنجرب قبل المتابعة (**Inserisci il numero** = أدخل العدد):

```
Inserisci il numero: -5
Inserisci il numero: 0
Inserisci il numero: 15
Inserisci il numero: 42
Inserisci il numero: 5
```

مع -5 و0 و15 و42 يتكرر السؤال؛ ومع 5 يتابع البرنامج. ويمكن تجربة باقي الأعداد من 1 إلى 9. المشكلة الأولى، الحصول على المعلومة، حُلت.

ملاحظة

يمكن كتابة الشرط نفسه أيضًا بلا NOT، بالقول مباشرة متى يكون العدد خاطئًا: `N < 1 || N > 9` (أصغر من 1 **أو** أكبر من 9). الكتابتان **متكافئتان**.

5.2 قاعدة: كم بعدًا، كم for

تعريف

عند رسم شيء، يكون **عدد** **أبعاد** الرسم مساويًا **لعدد** **for** **المتداخلة**:

- رسم على **خط** (بعد واحد)، مثل سطر التمرين 1: **for** **واحدة**؛
- رسم على **مستوٍ** (بعدان): مربع، مثلث، دائرة، بيت صغير مع شجرة: **for** **مرتين**، إحداهما داخل الأخرى؛
- جسم من **العالم** **الحقيقي** (3 أبعاد): **for** **ثلاث** **مرات**، `for` داخل `for` داخل `for` أخرى.

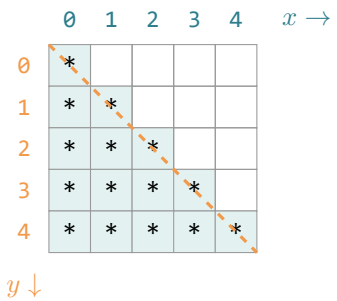
ليست حيلة: إنها **حقيقة** تصلح لكل الرسوم.

المثلث رسم ببعدين: نحتاج إلى **for** **متداخلتين**، ببنية المستطيل نفسها، بما في ذلك `printf("\n");` في نهاية **for** **الداخلية**.

5.3 من المربع إلى المثلث

الخطوة **الأولى** هي المربع الممتلئ: `for` مرتين، كلتاهما حتى `N`. مع 5 يطبع البرنامج مربعًا 5 في 5 من النجوم. ليس مثلثًا بعد، لكننا قريبون.

لننظر الآن إلى الإحداثيات، كما في المستطيل الفارغ: الأعمدة x من 0 إلى 4، والأسطر y من 0 إلى 4.



متى تُطبع النجمة؟ عندما نكون **تحت** **القطر** (أو عليه). لنأخذ السطر y = 2: النجوم 3، في الأعمدة 0 و1 و2. والنقطة على القطر لها x = 2 وy = 2. وبما أننا نسير أفقيًا في الحلقة الداخلية، ففي السطر y تُطبع النجوم ما

دام x **أصغر** **من** **أو** **يساوي** y. إذن **لا** **تصل** الحلقة الداخلية **إلى** **N**، بل تتوقف عند x <= y: يتغير فقط شرط for الداخلية.

```c
#include <stdio.h>

int main()
{
    int N;

    // أكرر إذا لم يكن العدد أكبر من 0 وأصغر من 10
    do
    {
        printf("Inserisci il numero: ");
        scanf("%d", &N);
    } while (!(N > 0 && N < 10));

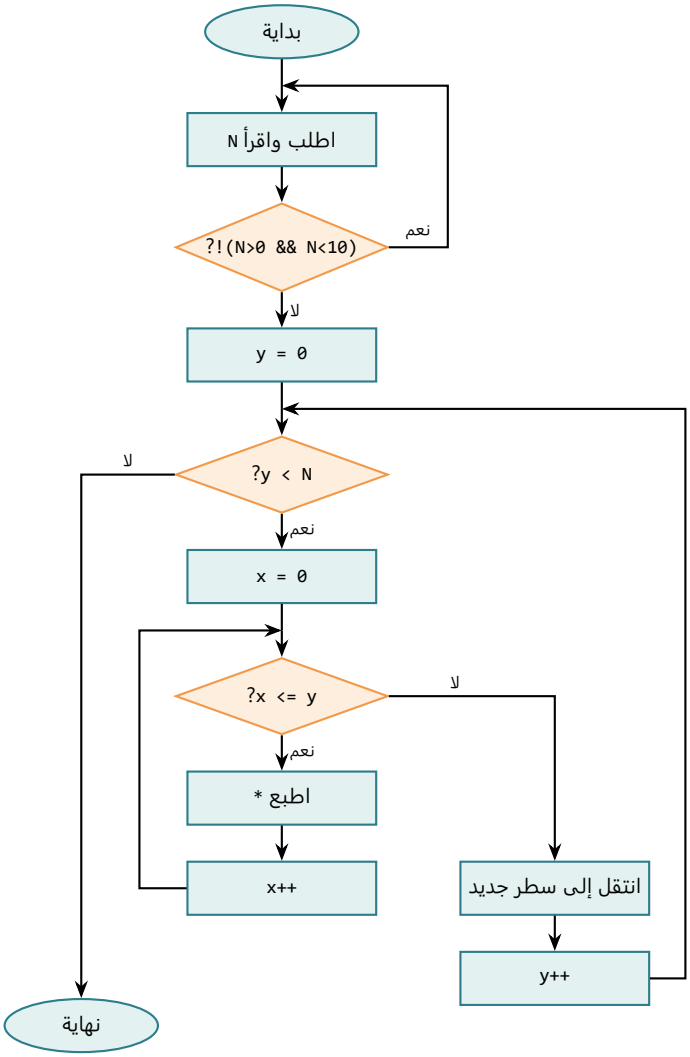
    // متداخلتان for: رسم ببعدين
    for (int y = 0; y < N; y++)
    {
        // أطبع النجوم ما دام x <= y في السطر
        for (int x = 0; x <= y; x++)
        {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

```
Inserisci il numero: 5
*
**
***
****
*****
```

ملاحظة

هذا هو البرنامج المكتوب في القسم في آخر دقائق الدرس، والمجرب بـ5. حاول إعادة بنائه وحدك، ابتداءً من المربع الممتلئ، قبل النظر إليه.



شكل 4: التمرين 4: التحقق من المعلومة بـNOT، ثم الحلقتان المتداخلتان. مقارنة بالمستطيل يتغير فقط شرط الحلقة الداخلية: x <= y.

6 ما يجب تذكره من هذا الدرس

- **لطلب** **معلومة** وإعادة السؤال إذا كانت خاطئة: **do** **while**، وفي شرطها **متى** **نكرر** (المعلومة الخاطئة). "أكبر من 0" تعني أننا نكرر مع `N > 1`.
- للتكرار عندما **لا** تحترم معلومة شرطًا مركبًا: نكتب الشرط الصحيح بـ**AND** (&&) وننفيه بـ**NOT** (!). "و" هي AND، و"أو" هي OR.
- لكل نوع بيانات **حد**: `int` من 32 بت يمتد من نحو -2 إلى $+2$ مليار؛ وبعده يوجد **overflow**.
- المسألة الصعبة **تُقسم** **إلى** **أجزاء**: أولًا سطر، ثم السطر مكررًا.
- `for` داخل `for` هي **for** **متداخلة**: **الخارجية** تعد الأسطر (y)، و**الداخلية** الأعمدة (x)، **بمتغيرين** **مختلفين**. و**السطر** **الجديد** يوضع بعد الحلقة الداخلية.
- **أبعاد** الرسم = عدد **for** **المتداخلة**.
- لاختيار **ما** **يُطبع** في كل نقطة نستخدم **الإحداثيات** x وy، التي تبدأ من 0: السطر الأخير `R - 1`، والعمود الأخير `C - 1`. وعندما لا يُطبع الحرف نحتاج إلى **else** مع **الفراغ**.
- الكود **المزاج** يسمح بإيجاد الأخطاء فورًا؛ ويُجرَّب البرنامج **في** **كل** **خطوة**.
- يجب أن يفعل البرنامج **ما** **يطلبه** **النص**.

> **ملاحظة**
>
> **واجب منزلي.** تجد على Classroom **الواجب** **C01**: أشكال أخرى بالحلقات المتداخلة (المثلث المقلوب، مثلث الأعداد، المثلث إلى اليمين، X)، وعد تنازلي بـ`while` وقائمة بـ`switch`. نكمل من حيث توقفنا في القسم.

7 مسرد: كلمات إيطالية تُستخدم في القسم

- **asterisco**: نجمة
- **riga / colonna**: سطر / عمود
- **larghezza / altezza**: عرض / ارتفاع
- **rettangolo pieno / vuoto**: مستطيل ممتلئ / فارغ
- **bordo**: حافة
- **spazio**: فراغ
- **annidato**: متداخل
- **ciclo esterno / interno**: حلقة خارجية / داخلية
- **triangolo rettangolo**: مثلث قائم
- **diagonale**: قطر
- **a capo**: إلى سطر جديد
- **battaglia navale**: لعبة معركة السفن (تُلعب على الورق)